Device Configuration

Configuring Your Device

Learn how to set up and customize your T3000 device settings.

Where to Find Settings

1. Open your device in T3000
2. Click on the **Settings** tab
3. You'll see all configuration options

Basic Setup

Give Your Device a Name:

- Find "Device Name" field
- Type a friendly name (like "Main Lobby Controller")
- This helps you identify it later

Set the Location:

- Enter where the device is installed
- Examples: "Building A - 2nd Floor", "Mechanical Room"

Network Connection

Setting IP Address:

Your device needs an IP address to communicate on the network.

- **IP Address**: Like a phone number for your device (e.g., 192.168.1.100)
- **Subnet Mask**: Usually 255.255.255.0 (ask your IT department if unsure)
- **Gateway**: The router address (usually ends in .1, like 192.168.1.1)

Tip: Write down these settings! You'll need them if you ever reset the device.

Time Settings

Keep Your Device's Clock Accurate:

1. **Time Zone**: Select your local timezone
2. **NTP Server** (automatic time): Enter `pool.ntp.org` or ask IT
3. **Sync Interval**: Set to 24 hours for daily updates

Why time matters:

- Schedules run at correct times
- Trend logs show accurate timestamps
- Alarms are properly timestamped

What Features to Enable

Check these boxes to turn features on:

- ☑ **Trend Logging** - Save historical data (recommended)
- ☑ **Alarms** - Get notified of problems (recommended)
- ☑ **Schedules** - Automate operations (if needed)
- ☑ **Remote Access** - Access from other locations (optional)

Saving Your Changes

Important Steps:

1. Make your changes
2. Click **Apply** - This tests your settings
3. Wait to see if device responds (usually 5-10 seconds)
4. If everything works, click **Save**

⚠ **Warning:** Some changes require the device to restart. You might lose connection for 1-2 minutes.

Backing Up Your Settings

Always backup before making big changes!

To Save Configuration:

1. Go to Settings > Backup
2. Click "Export Configuration"
3. Save the file somewhere safe (name it with today's date)

To Restore Configuration:

1. Go to Settings > Restore
2. Click "Choose File"
3. Select your backup file
4. Click "Import"

Quick Troubleshooting

Can't save settings?

- Check if device is online
- Make sure no one else is editing it
- Try refreshing the page

Device won't connect after changing IP?

- The IP address might be wrong
- Restore from backup
- Contact your network administrator

Getting Help

Need assistance with configuration? Switch to the **Technical** tab for advanced options and code examples.

Device Configuration API

Programmatically configure T3000 device settings via API.

Configuration Object Structure

TYPESCRIPT

```typescript
interface DeviceConfig {
  // Basic Information
  deviceName: string;
  description: string;
  location: string;
  firmwareVersion: string;  // read-only
  model: string;            // read-only

  // Network Configuration
  network: {
    ipAddress: string;
    subnetMask: string;
    gateway: string;
    dnsServer: string;
    dhcpEnabled: boolean;
  };

  // Protocol Settings
  bacnet: {
    deviceInstance: number;  // 0-4194303
    networkNumber: number;
    macAddress: string;
    apduTimeout: number;     // milliseconds
    apduRetries: number;
  };

  modbus: {
    slaveId: number;         // 1-247
    baudRate: number;        // 9600, 19200, 38400, 57600, 115200
    parity: 'none' | 'even' | 'odd';
    stopBits: 1 | 2;
  };

  // Time Synchronization
  time: {
    timezone: string;
    ntpServer: string;
    syncInterval: number;    // hours
```

```
  };

  // Communication Settings
  communication: {
    pollingInterval: number;      // seconds
    connectionTimeout: number;    // seconds
    maxConcurrentRequests: number;
    autoReconnect: boolean;
  };

  // Feature Flags
  features: {
    trendLogging: boolean;
    alarms: boolean;
    schedules: boolean;
    remoteAccess: boolean;
  };
}
```

Get Configuration

TYPESCRIPT

```
async function getDeviceConfig(serial: number): Promise<DeviceConfig> {
  const response = await fetch(
    `http://localhost:9103/api/t3_device/devices/${serial}/config`
  );

  if (!response.ok) {
    throw new Error(`Failed to get config: ${response.statusText}`);
  }

  return await response.json();
}
```

Update Configuration

TYPESCRIPT

```typescript
async function updateDeviceConfig(
  serial: number,
  config: Partial<DeviceConfig>
): Promise<void> {
  const response = await fetch(
    `http://localhost:9103/api/t3_device/devices/${serial}/config`,
    {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(config)
    }
  );

  if (!response.ok) {
    throw new Error(`Failed to update config: ${response.statusText}`);
  }
}

// Example: Update device name and location
await updateDeviceConfig(237219, {
  deviceName: 'Main_Lobby_Controller',
  location: 'Building A - Lobby'
});
```

Batch Configuration

Update multiple devices with same settings:

TYPESCRIPT

```typescript
async function batchUpdateConfig(
  serials: number[],
  config: Partial<DeviceConfig>
): Promise<void> {
  const promises = serials.map(serial =>
    updateDeviceConfig(serial, config)
  );

  await Promise.all(promises);
}

// Apply NTP settings to all devices
await batchUpdateConfig(
  [237219, 237220, 237221],
  {
    time: {
      ntpServer: 'pool.ntp.org',
      syncInterval: 24,
      timezone: 'America/New_York'
    }
  }
);
```

Configuration Backup/Restore

Export Configuration:

TYPESCRIPT

```typescript
async function exportConfig(serial: number): Promise<string> {
  const config = await getDeviceConfig(serial);
  const timestamp = new Date().toISOString().split('T')[0];
  const filename = `device_${serial}_config_${timestamp}.json`;

  // Create blob and download
  const blob = new Blob([JSON.stringify(config, null, 2)], {
    type: 'application/json'
  });

  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = filename;
  a.click();
  URL.revokeObjectURL(url);

  return filename;
}
```

Import Configuration:

TYPESCRIPT

```typescript
async function importConfig(
  serial: number,
  configFile: File
): Promise<void> {
  const text = await configFile.text();
  const config = JSON.parse(text) as DeviceConfig;

  // Remove read-only fields
  delete config.firmwareVersion;
  delete config.model;

  await updateDeviceConfig(serial, config);
}
```

Validation

Validate configuration before applying:

TYPESCRIPT

```typescript
function validateConfig(config: Partial<DeviceConfig>): string[] {
  const errors: string[] = [];

  // Validate BACnet device instance
  if (config.bacnet?.deviceInstance !== undefined) {
    if (config.bacnet.deviceInstance < 0 ||
        config.bacnet.deviceInstance > 4194303) {
      errors.push('BACnet device instance must be 0-4194303');
    }
  }

  // Validate Modbus slave ID
  if (config.modbus?.slaveId !== undefined) {
    if (config.modbus.slaveId < 1 || config.modbus.slaveId > 247) {
      errors.push('Modbus slave ID must be 1-247');
    }
  }

  // Validate IP address format
  if (config.network?.ipAddress) {
    const ipRegex = /^(\d{1,3}\.){3}\d{1,3}$/;
    if (!ipRegex.test(config.network.ipAddress)) {
      errors.push('Invalid IP address format');
    }
  }

  return errors;
}

// Usage
const config = { /* ... */ };
const errors = validateConfig(config);
if (errors.length > 0) {
  console.error('Configuration errors:', errors);
} else {
  await updateDeviceConfig(serial, config);
}
```

Configuration Templates

Predefined configuration templates:

TYPESCRIPT

```typescript
const configTemplates = {
  standard: {
    communication: {
      pollingInterval: 30,
      connectionTimeout: 5,
      maxConcurrentRequests: 10,
      autoReconnect: true
    },
    features: {
      trendLogging: true,
      alarms: true,
      schedules: true,
      remoteAccess: false
    }
  },

  highPerformance: {
    communication: {
      pollingInterval: 5,
      connectionTimeout: 2,
      maxConcurrentRequests: 20,
      autoReconnect: true
    }
  },

  lowBandwidth: {
    communication: {
      pollingInterval: 60,
      connectionTimeout: 10,
      maxConcurrentRequests: 5,
      autoReconnect: true
    }
  }
};

// Apply template
await updateDeviceConfig(serial, configTemplates.standard);
```

Configuration Change Events

Listen for configuration changes:

```typescript
const ws = new WebSocket('ws://localhost:9103/ws');

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);

  if (data.type === 'config_changed') {
    console.log('Configuration updated:', {
      serial: data.serial,
      field: data.field,
      oldValue: data.oldValue,
      newValue: data.newValue,
      timestamp: data.timestamp
    });
  }
};

// Subscribe to config changes
ws.send(JSON.stringify({
  type: 'subscribe_events',
  filters: { types: ['config_changed'] }
}));
```

Best Practices

1. **Validate Before Apply**
   - Always validate configuration objects
   - Check for conflicts (e.g., duplicate BACnet IDs)
   - Test in development environment first
2. **Atomic Updates**
   - Use transactions when updating related settings
   - Rollback on partial failure
   - Keep backup of previous configuration
3. **Change Management**

- Log all configuration changes with timestamp
- Include user/system that made change
- Document reason for change

4. **Error Handling**

```typescript
try {
  await updateDeviceConfig(serial, config);
} catch (error) {
  // Rollback to previous config
  await importConfig(serial, backupFile);
  console.error('Config update failed, rolled back:', error);
}
```

Next Steps

- Device Monitoring - Monitor configured devices
- Troubleshooting - Resolve configuration issues
- REST API - Full API documentation